

DegreeBaba — AI Intern Task Brief

Content Parsing Micro App: Word → ACF Field Mapping

Issued by: DegreeBaba (Degreebaba Pvt Ltd) **Date:** May 2026 **Task Type:** Research + Prototype (MVP-level implementation)

1. Background

DegreeBaba is an EdTech aggregator platform that lists university programs across India. The website runs on WordPress with ACF Pro (Advanced Custom Fields) to power structured content pages — University Pages, Course Pages, and Specialization Pages.

We have a large content pipeline: ~50 universities, ~250 courses, ~250 specializations. Content writers produce Word (.docx) files for each page. These files need to be parsed and published to WordPress in a structured format without data loss.

The objective of this task is to build a **Content Publishing Micro App** — an internal tool that takes a Word file as input and correctly populates all the right ACF fields on the right WordPress page type.

2. What You'll Receive

You will be given the following materials at the start of the task:

#	Asset	Description
1	NMIMS_University_Page.html	Final HTML design of the University page
2	NMIMS_Online_MBA.html	Final HTML design of the Course page
3	NMIMS_MBA_Marketing.html	Final HTML design of the Specialization page
4	University_Sample.docx	Sample Word file for a University page
5	Course_Sample.docx	Sample Word file for a Course page
6	Specialization_Sample.docx	Sample Word file for a Specialization page
7	Micro App UI Design Templates	Screen-by-screen UI designs for the micro app (8 screens)
8	ACF Field Definitions	Complete list of ACF fields for all 3 page types (~55 University, ~60 Course, ~58 Specialization)

Important: The HTML files are the source of truth for what sections exist on each page and what data needs to populate each section. The Word files are how content writers structure that data. Your job is to reliably bridge the two.

3. The Core Problem

Each Word file has sections with headings (e.g. "About the University", "Key Highlights", "Programs Offered", etc.). Each HTML page has corresponding sections that pull content from specific ACF fields in WordPress.

The challenge is: **how do you parse a Word file and correctly map every piece of content to its exact ACF field — without missing any data, duplicating values, or mapping content to the wrong field?**

Why This Is Non-Trivial

- Word files use natural language headings, not technical field names. E.g. a writer writes "Why Choose NMIMS?" but the ACF field is `why_choose_heading`.
- Some sections are simple text; others are complex structured data (tables, lists, repeaters, FAQs, review cards, program fee tables).
- Some fields are derived/extracted from prose (hero stats like student count, faculty count, hiring partners).
- Heading names vary slightly across universities (e.g. "Fees & EMI" vs "Fee Structure").
- Data loss is the worst outcome — a missing field means a broken section on the live website.

Known Failure Patterns (From Prior Testing)

Based on testing with 5 universities, these failure patterns were observed:

- AI confused `stat_faculty` with `num_programs` (similar numbers appearing in the same paragraph)
 - Missing stats when the Word file didn't state them explicitly
 - Duplicate values across two separate stat fields
 - Thin content in body fields (AI returning only 1-2 lines when the Word file had full paragraphs)
 - Wrong JSON structure for repeater fields (flat string instead of array)
-

4. Your Objective

Design and implement a **working MVP** of the content parsing pipeline. Specifically:

1. **Parse** a Word file and extract all content sections with their headings

2. **Map** each section to the correct ACF field name for that page type (University / Course / Specialization)
3. **Structure** complex data correctly — JSON arrays for repeaters, WYSIWYG HTML for body fields, plain text for heading and stat fields
4. **Validate** the output — flag missing fields, thin content, or structural errors before pushing to WordPress
5. **Output** a publishable payload that can be sent to the WordPress REST API

The micro app does not need to be a polished product UI at this stage. A working pipeline with a simple interface (or even a CLI/notebook) counts as MVP.

5. What You Are Expected to Deliver

Phase 1 — Research & Design (present before building)

Present **2–3 approaches** to solving the Word → ACF mapping problem. For each approach, document:

- How it works (brief technical overview)
- Tools/libraries/AI models used
- Pros and cons
- Estimated reliability for the known failure patterns listed above
- Your recommended approach and why

Example approaches you might explore (not limited to these):

- **AI-only mapping:** Pass the full Word text to an LLM and ask it to return a structured JSON payload matching the ACF schema
- **Deterministic + AI hybrid:** Use heading keyword matching for most fields (deterministic), use AI only for ambiguous fields and stat extraction
- **Embedding similarity:** Embed Word headings and ACF field descriptions, match by cosine similarity
- **Structured template parsing:** Enforce a strict Word template (with a Quick Facts block and fixed headings) so parsing becomes mostly deterministic
- **Vision-based approach:** Convert Word to images or PDF, use a multimodal model to read layout and populate fields

Phase 2 — MVP Implementation

Build the recommended approach to MVP level:

- Accepts a Word file (.docx) as input
- Identifies the page type (University / Course / Specialization)
- Returns a structured JSON payload matching the ACF field schema

- Includes a validation report (fields mapped, fields missing, fields flagged as thin)

Tech stack is your choice — Python script, Node.js app, Streamlit, Jupyter notebook, a simple web UI, or a Bolt.new prototype. What matters is that it works reliably.

6. Tools & Constraints

Tools — You Have Full Freedom

Use any AI tool, library, or service you believe is right for the job:

- **AI models:** Claude API, OpenAI GPT-4o, Gemini, local models — anything
- **Parsing libraries:** Mammoth.js (Node), python-docx (Python), Apache POI, etc.
- **Frameworks:** Python, Node.js, React, Streamlit, Bolt.new, n8n, etc.
- **Prototyping environments:** Replit, Colab, Cursor, VS Code — your call

Hard Constraints

Constraint	Reason
Must handle WYSIWYG fields as HTML, not plain text	Elementor renders HTML in body fields — markdown or plain text breaks the layout
Repeater fields must output valid JSON arrays	WordPress REST API rejects malformed repeater payloads
Stat fields: numbers only, max 6 characters (e.g. 185K+)	Hero stats render in small badge cards
Zero data loss tolerance	Missing content = broken page section visible to students
Must not create a duplicate WordPress post on re-upload	Re-uploading an existing page must update (not duplicate) and reset to Draft

Reference Field Schema (Partial Example)

json

```
{
  "university_name": "NMIMS",
  "university_full_name": "Narsee Monjee Institute of Management Studies",
  "naac_grade": "A+",
  "ugc_status": "UGC Entitled",
  "established_year": "1981",
  "stat_students": "185K+",
  "stat_alumni": "82K+",
  "stat_hiring_partners": "20K+",
  "stat_faculty": "140+",
  "about_heading": "About NMIMS",
  "about_content": "<p>NMIMS is one of India's...</p>",
  "programs": [
    { "name": "MBA", "duration": "2 Years", "fee": "₹1,20,000/semester", "mode": "On"
  ],
  "faqs": [
    { "question": "Is NMIMS degree valid?", "answer": "Yes, it is UGC entitled..." }
  ]
}
```

The full ACF field definitions document (with all ~55/60/58 fields per page type) will be provided with your materials.

7. Evaluation Criteria

Your solution will be evaluated on:

Criterion	Weight
Completeness — how many fields are correctly populated from the Word file	35%
Accuracy — correct field mapping, no cross-contamination of values	25%
Structure — correct data types (HTML vs plain text vs JSON array)	20%
Approach quality — clarity of your research presentation and reasoning	10%
Code quality / reproducibility — can someone else run it on a new Word file?	10%

Scoring Benchmark (Based on Prior Testing)

Score	Interpretation
55-65 / 100	Basic extraction working, major structural issues

Score	Interpretation
65-75 / 100	Good coverage, some stat/repeater failures
75-85 / 100	Strong — acceptable for phased rollout
85+ / 100	Excellent — near production-ready

Prior MVP testing with a basic AI-only approach scored in the 72-78 range. Your target is **80+ on at least 2 of the 3 page types**.

8. Timeline

Milestone	Deadline
Review all materials, set up environment	Day 1 — Morning
Research: 2-3 approaches documented	Day 1 — Afternoon
Present approach recommendations (async doc or quick call)	Day 1 — EOD
MVP implementation	Day 2 — Morning to Afternoon
Testing on sample Word files per page type	Day 2 — Afternoon
Final submission: code + demo + brief writeup	Day 2 — EOD

9. Submission

Submit the following:

- Research document** (Google Doc, Notion, or PDF) — your 2-3 approach analysis + recommendation
- MVP code** (GitHub repo or shared folder) — runnable on a new Word file
- Test results** — JSON output for at least 2 Word files per page type (6 files total)
- Validation report** — which fields mapped, which were missing, any structural flags
- Brief writeup** (1 page max) — what you'd change if you had 2 more weeks

10. Additional Notes

- The HTML design files show you exactly how each section of the page looks visually. Use

them to understand *why* each ACF field exists — a field like `programs_intro` powers a subtitle line above a programs table, which tells you it should be a short 1-line summary, not a paragraph.

- The micro app UI design templates show the 8-screen product vision. You don't need to build all 8 screens — but understanding the full product intent will help you make better design decisions in the MVP.
- The re-upload behaviour is a **hard requirement**: if a Word file is uploaded for a page that already exists in WordPress, the app must update that existing post and set its status back to `draft`. Never create a new duplicate post.
- Think about the content writer experience. The more you can make parsing deterministic (by enforcing a structured Word template), the less you depend on AI reliability. A strict Quick Facts block at the top of every Word file has already been validated as effective for hero stats.
- If you propose an AI-dependent step, document what happens when the AI gets it wrong — what's your fallback or override mechanism?

Good luck. The goal is zero data loss and a tool the content team can use with confidence.

Questions? Reach out to the project lead directly.